motion_base

Release 1.0

Automated Script

Jun 20, 2025

CONTENTS

This documentation includes package dependencies, messages, services, actions, launch files, parameter files, robot descriptions, test code, images, topic graphs, and auto-generated Doxygen diagrams.

CHAPTER

ONE

DEPENDENCIES

Dependency
can_msgs
geometry_msgs
motion_rover_ros2_msgs
nav_msgs
rclcpp
rosidl_default_generators
rosidl_default_runtime
std_msgs
std_srvs
tf2_geometry_msgs
tf2_ros

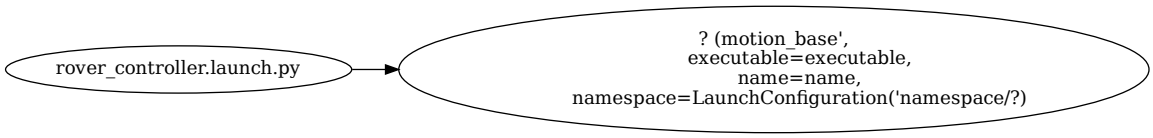
CHAPTER

TWO

LAUNCH FILES

2.1 rover_controller.launch.py

Nodes launched by this file (diagram):



Node Table:

Package	Executable	Name
motion_base', executable=executable, name=name, namespace=LaunchConfiguration(‘na	?	?

CHAPTER

THREE

DISCOVERED TOPICS

Topic Name	Message Type	Publishers	Subscribers
/canbus/tx	can_msgs::msg::Frame	camera_lift	
camera_lift/status	motion_rover_ros2_msgs::msg::CameraLiftStatus	camera_lift	
/canbus/rx	can_msgs::msg::Frame		camera_lift
camera_lift/target_height	std_msgs::msg::Float32		camera_lift
/canbus/tx	can_msgs::msg::Frame	rover_wheel	
odom	nav_msgs::msg::Odometry	rover_wheel	
/cmd_vel	geometry_msgs::msg::Twist		rover_wheel
/canbus/rx	can_msgs::msg::Frame		rover_wheel
/canbus/tx	can_msgs::msg::Frame	rover_lift	
rover_lift/status	motion_rover_ros2_msgs::msg::RoverLiftStatus	rover_lift	
/canbus/rx	can_msgs::msg::Frame		rover_lift
rover_lift/target_height	std_msgs::msg::Float64		rover_lift

CHAPTER

FOUR

PARAMETER FILES IN *CONFIG*

4.1 config/camera_lift.yaml

```yaml
# Node named 'rover_camera_lift' without namespace
rover_camera_lift:
  ros__parameters:
    can_tx_id: 288
    can_rx_id: 544
    max_height: 0.2
    tolerance: 0.005
    control_rate: 10.0

# Node named 'rover_camera_lift' under '/rover' namespace
rover:
  rover_camera_lift:
    ros__parameters:
      can_tx_id: 288
      can_rx_id: 544
      max_height: 0.2
      tolerance: 0.005
      control_rate: 10.0
```

4.2 config/rover_lift.yaml

```yaml
#rover_lift
# Parameters for node launched without a namespace
lift_control:
  ros__parameters:
    can_tx_id: 272              # 0x110
    can_rx_base_id: 528         # 0x210
    max_height: 0.16
    min_height: 0.0
    movement_timeout: 30.0
    height_tolerance: 0.0001
    default_speed: 10.0

# Parameters for node launched with namespace `/rover`
rover:
```

(continues on next page)

(continued from previous page)

```
  lift_control:
    ros__parameters:
      can_tx_id: 272
      can_rx_base_id: 528
      max_height: 0.16
      min_height: 0.0
      movement_timeout: 30.0
      height_tolerance: 0.001
      default_speed: 10.0
```

4.3 config/rover_wheel.yaml

```
# Node launched with name 'rover_wheel_node' without namespace
rover_wheel_node:
  ros__parameters:
    wheel_radius: 0.05
    lx: 0.15
    ly: 0.15

    wheel_vel_scale: 1000.0
    big_endian: false
    can_id_wheel_vel: 256
    can_id_control: 257
    can_id_feedback: 512

    use_vel_smoothing: true
    accel_smooth_duration: 0.3
    decel_smooth_duration: 0.3
    cmd_vel_timeout: 0.5
    vel_threshold: 0.0005
    control_rate: 10.0

# Node launched with name 'rover_wheel_node' under namespace '/rover'
rover:
  rover_wheel_node:
    ros__parameters:
      wheel_radius: 0.05
      lx: 0.15
      ly: 0.15

      wheel_vel_scale: 1000.0
      big_endian: false
      can_id_wheel_vel: 256
      can_id_control: 257
      can_id_feedback: 512

      use_vel_smoothing: true
      accel_smooth_duration: 0.3
      decel_smooth_duration: 0.3
      cmd_vel_timeout: 0.5
```

(continues on next page)

(continued from previous page)

```
    vel_threshold: 0.0005
    control_rate: 10.0
```

CHAPTER

FIVE

TEST FILES IN *TEST*

Listing 1: (first 20 lines, see file for full code)

```
#include <gtest/gtest.h>
#include <rclcpp/rclcpp.hpp>
#include <geometry_msgs/msg/twist.hpp>
#include <nav_msgs/msg/odometry.hpp>
#include <std_srvs/srv/trigger.hpp>
#include <std_srvs/srv/set_bool.hpp>
#include "motion_base/rover_wheel_node.hpp"

class RoverWheelNodeTest : public ::testing::Test {
protected:
    void SetUp() override {
        rclcpp::init(0, nullptr);
        node = std::make_shared<RoverWheelNode>();
    }

    void TearDown() override {
        node.reset();
        rclcpp::shutdown();
    }
```

Listing 2: (first 20 lines, see file for full code)

```
#include <gtest/gtest.h>
#include "motion_base/rover_wheel_node.hpp"

// =============== MecanumKinematics Tests ===============
TEST(MecanumKinematicsTest, Inverse_ForwardOnly) {
    MecanumKinematics kin(0.05, 0.15, 0.15);
    std::array<double, 3> cmd_vel = {1.0, 0.0, 0.0}; // Forward
    auto wheel_vel = kin.inverse(cmd_vel);

    EXPECT_NEAR(wheel_vel[0], 20.0, 1e-6); // FL
    EXPECT_NEAR(wheel_vel[1], 20.0, 1e-6); // FR
    EXPECT_NEAR(wheel_vel[2], 20.0, 1e-6); // BL
    EXPECT_NEAR(wheel_vel[3], 20.0, 1e-6); // BR
}

TEST(MecanumKinematicsTest, Inverse_StrafeRight) {
```

(continues on next page)

(continued from previous page)

```
    MecanumKinematics kin(0.05, 0.15, 0.15);
    std::array<double, 3> cmd_vel = {0.0, -1.0, 0.0}; // Strafe right
    auto wheel_vel = kin.inverse(cmd_vel);
```

CHAPTER

SIX

DOXYGEN DIAGRAMS AND CLASS HIERARCHIES

class **CameraLiftNode** : public *rclcpp*::Node

#include <camera_lift_node.hpp> ROS 2 node for controlling and monitoring a camera lift mechanism.

Main ROS 2 node for controlling the camera lift.

Provides both action and topic interfaces for control, and publishes lift status.

This node handles action goals and topic commands to move the camera lift, sends CAN commands, receives feedback, and manages goal states.

Public Functions

CameraLiftNode()

Constructor. Initializes parameters, publishers, subscribers, and action server.

Private Functions

void **declareParameters**()

Declare all configurable parameters with default values.

Declare ROS parameters with default values.

void **loadParameters**()

Load parameters from the parameter server.

Load parameters from the ROS parameter server.

void **validateParameters**()

Validate and correct parameters if necessary.

Validate loaded parameters and reset to defaults if invalid.

rclcpp_action::GoalResponse **handle_goal**(const rclcpp_action::GoalUUID&, std::shared_ptr<const *CameraLiftAction*::Goal>)

Action server goal callback.

Handle new action goal requests.

Always accept the goal for further validation in handle_accepted.

Parameters

- **uuid** – Goal UUID.
- **goal** – Goal message.
- **uuid** – **[in]** The goal UUID.
- **goal** – **[in]** The goal message.

Returns
GoalResponse indicating acceptance or rejection.

Returns
Always ACCEPT_AND_EXECUTE.

rclcpp_action::CancelResponse **handle_cancel**(const std::shared_ptr<*GoalHandleCameraLift*>)

Action server cancel callback.

Handle action goal cancellation requests.

Parameters

- **goal_handle** – Handle to the goal being canceled.
- **goal_handle** – **[in]** The handle to the goal being cancelled.

Returns
CancelResponse indicating acceptance.

Returns
Always ACCEPT.

void **handle_accepted**(const std::shared_ptr<*GoalHandleCameraLift*>)

Action server goal acceptance callback.

Accept and execute a valid action goal.

Validates the requested height, checks for active goals, and sends the height command if valid.

Parameters

- **goal_handle** – Handle to the accepted goal.
- **goal_handle** – **[in]** The handle to the accepted goal.

void **send_height_command**(double height_m)

Send a CAN command to move the camera lift to the specified height.

Send a height command to the hardware via CAN.

Converts the height from meters to millimeters and sends it in the CAN message payload.

Parameters

- **height_m** – Target height in meters.
- **height_m** – **[in]** Target height in meters.

void **canRxCallback**(const can_msgs::msg::Frame::SharedPtr msg)

CAN RX callback. Updates current height and publishes status.

Callback for CAN RX messages.

Updates the current height based on hardware feedback and publishes a status message.

Parameters

- **msg** – CAN frame message.
- **msg** – **[in]** The received CAN frame.

void **topic_height_callback**(const std_msgs::msg::Float32::SharedPtr msg)

Topic subscriber callback for manual/slider control.

Callback for topic-based height commands.

Validates the requested height and starts a topic-based goal if valid.

Parameters

- **msg** – Float32 message with target height (meters).
- **msg** – **[in]** The received Float32 message (target height in meters).

void **control_loop**()

Main control loop. Handles action and topic goals, sends commands, and publishes feedback.

Main control loop.

Periodically checks and manages both action and topic goals, sending height commands and publishing feedback as needed.

Private Members

uint32_t **can_tx_id_**

CAN TX ID for sending height commands.

uint32_t **can_rx_id_**

CAN RX ID for receiving feedback.

double **max_height_**

Maximum allowed height (meters)

double **tolerance_**

Tolerance for goal achievement (meters)

double **control_rate_**

Control loop rate (Hz)

rclcpp::Subscription<can_msgs::msg::Frame>::SharedPtr **can_rx_sub_**

CAN RX subscriber.

rclcpp::Publisher<can_msgs::msg::Frame>::SharedPtr **can_tx_pub_**

CAN TX publisher.

rclcpp::Publisher<motion_rover_ros2_msgs::msg::CameraLiftStatus>::SharedPtr **feedback_pub_**

Status publisher.

rclcpp_action::Server<*CameraLiftAction*>::SharedPtr **action_server_**

Action server.

rclcpp::TimerBase::SharedPtr **control_timer_**

Control loop timer.

rclcpp::Subscription<std_msgs::msg::Float32>::SharedPtr **height_cmd_sub_**

Topic command subscriber.

std::atomic<double> **current_height_** = {0.0}

Current lift height (meters)

std::shared_ptr<*GoalHandleCameraLift*> **active_goal_**

Currently active action goal.

std::mutex **goal_mutex_**

Mutex for goal state.

std::atomic<bool> **topic_goal_active_** = {false}

Is a topic-based goal active?

std::atomic<double> **topic_goal_height_** = {0.0}

Target height from topic (meters)

class **GoalActiveGuard**

#include <rover_lift_node.hpp> RAII helper to automatically set and clear a boolean flag.

RAII helper to manage the command_in_progress_ flag.

Used to manage the state of command_in_progress_ in a thread-safe way.

Public Functions

explicit **GoalActiveGuard**(std::atomic<bool> &flag)

Constructor. Sets the referenced flag to true.

Parameters

flag – Reference to the atomic flag to manage.

~GoalActiveGuard()

Destructor. Sets the referenced flag to false.

Private Members

std::atomic<bool> &**flag_**

Reference to the atomic flag being managed.

class **MecanumKinematics**

#include <rover_wheel_node.hpp> Provides forward and inverse kinematics for a mecanum drive robot.

Public Functions

MecanumKinematics(double wheel_radius, double lx, double ly)

std::array<double, 4> **inverse**(const std::array<double, 3> &cmd_vel) const

std::array<double, 3> **forward**(const std::array<double, 4> &wheel_vel) const

Private Members

double **wheel_radius_**

double **lx_**

double **ly_**

double **lx_plus_ly_**

class **RoverLiftNode** : public *rclcpp*::Node

#include <rover_lift_node.hpp> ROS2 node for controlling and monitoring a rover's lift via CAN bus.

This node provides an action server interface for lift commands, subscribes to CAN bus messages, and publishes status updates. It manages concurrency, parameter validation, and feedback.

Public Functions

RoverLiftNode()

Construct the *RoverLiftNode* and initialize all parameters, publishers, and subscriptions.

Construct the *RoverLiftNode*, initialize parameters, publishers, and subscriptions.

Private Functions

void **declareParameters**()

Declare all configurable ROS2 parameters with default values.

Declare all configurable ROS2 parameters.

void **loadParameters**()

Load parameters from the ROS2 parameter server.

void **validateParameters**()

Validate loaded parameters for correctness.

Validate loaded parameters.

Throws
`std::runtime_error` – if any parameter is invalid.

rclcpp_action::GoalResponse **handle_goal**(const rclcpp_action::GoalUUID&, std::shared_ptr<const *RoverLiftAction*::Goal> goal)

Callback to handle incoming action goals.

Handle new action goal requests.

Parameters

- **uuid** – The unique identifier for the goal.
- **goal** – The goal message.

Returns
Goal response (accept or reject).

rclcpp_action::CancelResponse **handle_cancel**(const std::shared_ptr<*GoalHandleRoverLift*>)

Callback to handle action goal cancellation requests.

Handle action goal cancellation requests.

Parameters
goal_handle – The handle to the goal.

Returns
Cancel response (accept or reject).

void **handle_accepted**(const std::shared_ptr<*GoalHandleRoverLift*> goal_handle)

Callback when a new goal is accepted. Launches execution in a new thread.

Handle accepted action goals.

Parameters
goal_handle – The handle to the accepted goal.

void **execute**(const std::shared_ptr<*GoalHandleRoverLift*> goal_handle)

The main execution loop for a lift action goal.

Execute the lift action.

Parameters
goal_handle – The handle to the action goal.

double **getAverageHeight**()

Compute the average height across all lifts.

Compute the average lift height from all lifts.

Returns
The average height in meters.

void **canRxCallback**(const can_msgs::msg::Frame::SharedPtr msg)

Callback for incoming CAN bus messages.

Parameters
msg – The received CAN frame.

void **publishLiftStatus**()

Publish the current lift status to the appropriate topic.

Publish the current lift status.

void **handleLiftCommand**(double height)

Handle direct height commands from a topic (e.g., UI slider).

Handle direct height commands from a topic.

Parameters
height – The target lift height in meters.

bool **canAcceptNewCommand**()

Check if a new command can be accepted (i.e., no command in progress or the last command is finished).

Check if a new command can be accepted.

Returns
True if a new command can be accepted.

Returns
True if no command is in progress or the last command is finished.

void **finishCommandIfArrived**()

Mark the command as finished if the target height has been reached.

Mark command as finished if the target height is reached.

Private Members

uint32_t **can_tx_id_**

CAN bus transmit ID for sending commands to the lift controller.

uint32_t **can_rx_base_id_**

Base CAN bus receive ID for receiving lift state feedback.

double **max_height_**

Maximum allowable lift height (meters).

double **min_height_**

Minimum allowable lift height (meters).

double **movement_timeout_**

Timeout for reaching a target height (seconds).

double **height_tolerance_**

Acceptable error for reaching the target height (meters).

double **default_speed_**

Default lift movement speed (mm/s).

rclcpp::Publisher<can_msgs::msg::Frame>::SharedPtr **can_tx_pub_**

Publishes CAN frames to the bus.

rclcpp::Subscription<can_msgs::msg::Frame>::SharedPtr **can_rx_sub_**

Subscribes to CAN frames from the bus.

rclcpp::Publisher<motion_rover_ros2_msgs::msg::RoverLiftStatus>::SharedPtr **lift_status_pub_**

Publishes lift status messages.

rclcpp_action::Server<*RoverLiftAction*>::SharedPtr **action_server_**

Action server for lift commands.

rclcpp::Subscription<std_msgs::msg::Float64>::SharedPtr **lift_command_sub_**

Subscribes to direct height commands (e.g., from a UI slider).

std::array<double, *NUM_LIFTS*> **lift_heights_**

Most recent measured heights for each lift (meters).

std::array<*rclcpp*::Time, *NUM_LIFTS*> **lift_last_update_times_**

Time of last update for each lift.

std::mutex **height_mutex_**

Mutex to protect access to lift_heights_.

std::mutex **last_update_mutex_**

Mutex to protect access to lift_last_update_times_.

std::atomic<bool> **command_in_progress_** = {false}

Indicates if a lift command is currently being executed.

double **last_commanded_height_** = {0.0}

The most recent target height commanded.

std::mutex **command_mutex_**

Protects command state variables.

Private Static Attributes

static constexpr size_t **NUM_LIFTS** = 4

Number of independent lift mechanisms supported.

class **RoverWheelNode** : public *rclcpp*::Node

#include <rover_wheel_node.hpp> Main ROS 2 node for mecanum rover base control.

Public Functions

RoverWheelNode()

~RoverWheelNode()

Private Functions

void **declareParameters**()

void **loadParameters**()

void **validateParameters**()

void **sendWheelVelocities**(const std::array<double, 4> &wheel_ang_vel)

void **sendWheelEnable**(bool enable)

void **publishOdometry**(const *rclcpp*::Time &stamp, double vx, double vy, double wz)

void **cmdVelCallback**(const geometry_msgs::msg::Twist::SharedPtr msg)

void **enableCmdVelCallback**(const std::shared_ptr<std_srvs::srv::SetBool::Request> req,
std::shared_ptr<std_srvs::srv::SetBool::Response> res)

void **hardStopCallback**(const std::shared_ptr<std_srvs::srv::Trigger::Request> req,
std::shared_ptr<std_srvs::srv::Trigger::Response> res)

void **resetOdometryCallback**(const std::shared_ptr<std_srvs::srv::Trigger::Request> req,
std::shared_ptr<std_srvs::srv::Trigger::Response> res)

void **estopCallback**(const std::shared_ptr<std_srvs::srv::SetBool::Request> req,
std::shared_ptr<std_srvs::srv::SetBool::Response> res)

void **canRxCallback**(const can_msgs::msg::Frame::SharedPtr msg)

void **update**()

Private Members

double **wheel_radius_**

double **lx_**

double **ly_**

double **wheel_vel_scale_**

bool **use_vel_smoothing_**

double **accel_smooth_duration_**

double **decel_smooth_duration_**

double **cmd_vel_timeout_**

double **vel_threshold_**

bool **big_endian_**

uint32_t **can_id_wheel_vel_**

uint32_t **can_id_control_**

uint32_t **can_id_feedback_**

double **control_rate_**

Control loop rate (Hz)

std::unique_ptr<*MecanumKinematics*> **kinematics_**

std::unique_ptr<*VelocitySmoother*> **smoother_**

rclcpp::Subscription<geometry_msgs::msg::Twist>::SharedPtr **cmd_vel_sub_**

rclcpp::Subscription<can_msgs::msg::Frame>::SharedPtr **can_rx_sub_**

rclcpp::Publisher<can_msgs::msg::Frame>::SharedPtr **can_tx_pub_**

rclcpp::Publisher<nav_msgs::msg::Odometry>::SharedPtr **odom_pub_**

rclcpp::Service<std_srvs::srv::Trigger>::SharedPtr **reset_odom_srv_**

rclcpp::Service<std_srvs::srv::SetBool>::SharedPtr **enable_cmd_vel_srv_**

rclcpp::Service<std_srvs::srv::SetBool>::SharedPtr **estop_srv_**

std::unique_ptr<tf2_ros::TransformBroadcaster> **tf_broadcaster_**

rclcpp::TimerBase::SharedPtr **timer_**

rclcpp::TimerBase::SharedPtr **enable_timer_**

std::array<double, 3> **current_cmd_vel_**

std::array<double, 3> **target_cmd_vel_**

double **x_**

double **y_**

double **yaw_**

rclcpp::Time **last_time_**

rclcpp::Time **last_smooth_time_**

rclcpp::Time **last_cmd_vel_time_**

bool **wheels_enabled_**

bool **is_stopping_**

bool **cmd_vel_enabled_**

bool **estop_active_**

std::mutex **state_mutex_**

class **VelocitySmoother**

#include <rover_wheel_node.hpp> Provides cubic smoothing for velocity transitions (accel/decel).

Public Functions

VelocitySmoother(double accel_duration, double decel_duration)

std::array<double, 3> **smooth**(const std::array<double, 3> &current, const std::array<double, 3> &target, double elapsed, bool is_stopping) const

Public Static Functions

static double **smoothStep**(double start, double end, double t, double duration)

Private Members

double **accel_duration_**

double **decel_duration_**

namespace **rclcpp**

file **camera_lift_node.hpp**

#include "rclcpp/rclcpp.hpp"#include "rclcpp_action/rclcpp_action.hpp"#include "std_msgs/msg/float32.hpp"#include "can_msgs/msg/frame.hpp"#include "motion_rover_ros2_msgs/action/camera_lift.hpp"#include "motion_rover_ros2_msgs/msg/camera_lift_status.hpp"#include <atomic>#include <mutex>#include <memory> Declaration of the *CameraLiftNode* class for ROS 2 camera lift control.

Typedefs

using **CameraLiftAction** = motion_rover_ros2_msgs::action::CameraLift

using **GoalHandleCameraLift** = rclcpp_action::ServerGoalHandle<*CameraLiftAction*>

file **rover_lift_node.hpp**

#include "rclcpp/rclcpp.hpp"#include "rclcpp_action/rclcpp_action.hpp"#include "can_msgs/msg/frame.hpp"#include "motion_rover_ros2_msgs/action/rover_lift.hpp"#include "motion_rover_ros2_msgs/msg/rover_lift_status.hpp"#include "motion_rover_ros2_msgs/msg/lift_data.hpp"#include "std_msgs/msg/float64.hpp"#include <mutex>#include <array>#include <atomic>#include <thread>

Typedefs

using **RoverLiftAction** = motion_rover_ros2_msgs::action::RoverLift

using **GoalHandleRoverLift** = rclcpp_action::ServerGoalHandle<*RoverLiftAction*>

file **rover_wheel_node.hpp**

#include <rclcpp/rclcpp.hpp>#include <geometry_msgs/msg/twist.hpp>#include <nav_msgs/msg/odometry.hpp>#include <can_msgs/msg/frame.hpp>#include <std_srvs/srv/trigger.hpp>#include <std_srvs/srv/set_bool.hpp>#include <tf2_ros/transform_broadcaster.h>#include <array>#include <mutex>#include <memory> ROS 2 node for mecanum rover base control with CAN bus, velocity smoothing, odometry, and safety services.

This node provides:

- /cmd_vel subscriber with velocity smoothing
- CAN bus communication for wheel velocities and enable/disable
- Odometry publishing and TF
- /enable_cmd_vel, /hard_stop, /estop, /reset_odometry services
- Professional safety logic (E-stop lockout, smooth stopping, thread safety)

Author
Your Name

Date
2025-06-18

file **camera_lift.cpp**

*#include "*motion_base/camera_lift_node.hpp*"#include <cmath>*

Functions

int **main**(int argc, char **argv)

Main entry point.

Initializes and spins the *CameraLiftNode*.

file **motion_faults.cpp**

#include "rclcpp/rclcpp.hpp"

Functions

int **main**(int argc, char *argv[])

file **rover_lift.cpp**

*#include "*motion_base/rover_lift_node.hpp*"#include <cmath>#include <sstream>*

Functions

int **main**(int argc, char **argv)

Main entry point for the rover lift node.

Parameters

- **argc** – Argument count.
- **argv** – Argument vector.

Returns

Exit code.

file **rover_wheel.cpp**

*#include "*motion_base/rover_wheel_node.hpp*"#include <tf2/LinearMath/Quaternion.h>#include <tf2_geometry_msgs/tf2_geometry_msgs.hpp>#include <cmath>*

Functions

int **main**(int argc, char **argv)

dir **include**

dir **include/motion_base**

dir **src**

The above diagrams and class/function documentation are auto-generated by Doxygen and Graphviz.

6.1 camera_lift.cpp

Listing 1: (first 20 lines, see file for full code)

```
/**
 * @file camera_lift_node.cpp
 * @brief ROS 2 node for controlling a camera lift mechanism via CAN bus.
 *
 * This node provides both action and topic interfaces to command the camera lift
 * to a specific height. It communicates with the hardware using CAN messages.
 *
 * @author Your Name
```

(continues on next page)

(continued from previous page)

```
 * @date 2025-06-20
 */

#include "motion_base/camera_lift_node.hpp"
#include <cmath>

/**
 * @class CameraLiftNode
 * @brief Main ROS 2 node for controlling the camera lift.
 *
 * This node handles action goals and topic commands to move the camera lift,
 * sends CAN commands, receives feedback, and manages goal states.
```

6.2 rover_wheel.cpp

Listing 2: (first 20 lines, see file for full code)

```
/**
 * @file rover_wheel_node.cpp
 * @brief Implementation of RoverWheelNode for mecanum rover base control.
 */

#include "motion_base/rover_wheel_node.hpp"
#include <tf2/LinearMath/Quaternion.h>
#include <tf2_geometry_msgs/tf2_geometry_msgs.hpp>
#include <cmath>

// ==================== MecanumKinematics Implementation ====================

// Constructor: Initializes kinematics with wheel radius and robot dimensions
MecanumKinematics::MecanumKinematics(double wheel_radius, double lx, double ly)
    : wheel_radius_(wheel_radius), lx_(lx), ly_(ly), lx_plus_ly_(lx + ly)
{
    // Validate input parameters
    if (wheel_radius <= 0.0 || lx <= 0.0 || ly <= 0.0) {
        throw std::invalid_argument("MecanumKinematics: wheel_radius, lx, ly must be␣
↪positive");
    }
```

6.3 motion_faults.cpp

6.4 rover_lift.cpp

Listing 3: (first 20 lines, see file for full code)

```
/**
 * @file rover_lift_node.cpp
```

(continues on next page)

(continued from previous page)

```
 * @brief ROS2 node for rover lift control via CAN bus.
 */

#include "motion_base/rover_lift_node.hpp"
#include <cmath>
#include <sstream>

/**
 * @class GoalActiveGuard
 * @brief RAII helper to manage the command_in_progress_ flag.
 */
GoalActiveGuard::GoalActiveGuard(std::atomic<bool>& flag) : flag_(flag) { flag_ = true; }

GoalActiveGuard::~GoalActiveGuard() { flag_ = false; }

/**
 * @brief Construct the RoverLiftNode, initialize parameters, publishers, and␣
↪subscriptions.
 */
```

6.5 rover_lift_node.hpp

Listing 4: (first 20 lines, see file for full code)

```
#pragma once

#include "rclcpp/rclcpp.hpp"
#include "rclcpp_action/rclcpp_action.hpp"
#include "can_msgs/msg/frame.hpp"
#include "motion_rover_ros2_msgs/action/rover_lift.hpp"
#include "motion_rover_ros2_msgs/msg/rover_lift_status.hpp"
#include "motion_rover_ros2_msgs/msg/lift_data.hpp"
#include "std_msgs/msg/float64.hpp"
#include <mutex>
#include <array>
#include <atomic>
#include <thread>

using RoverLiftAction = motion_rover_ros2_msgs::action::RoverLift;
using GoalHandleRoverLift = rclcpp_action::ServerGoalHandle<RoverLiftAction>;

/**
 * @brief RAII helper to automatically set and clear a boolean flag.
 *
```

6.6 camera_lift_node.hpp

Listing 5: (first 20 lines, see file for full code)

```
/**
 * @file camera_lift_node.hpp
 * @brief Declaration of the CameraLiftNode class for ROS 2 camera lift control.
 */

#pragma once

#include "rclcpp/rclcpp.hpp"
#include "rclcpp_action/rclcpp_action.hpp"
#include "std_msgs/msg/float32.hpp"
#include "can_msgs/msg/frame.hpp"
#include "motion_rover_ros2_msgs/action/camera_lift.hpp"
#include "motion_rover_ros2_msgs/msg/camera_lift_status.hpp"
#include <atomic>
#include <mutex>
#include <memory>

using CameraLiftAction = motion_rover_ros2_msgs::action::CameraLift;
using GoalHandleCameraLift = rclcpp_action::ServerGoalHandle<CameraLiftAction>;
```

6.7 rover_wheel_node.hpp

Listing 6: (first 20 lines, see file for full code)

```
/**
 * @file rover_wheel_node.hpp
 * @brief ROS 2 node for mecanum rover base control with CAN bus, velocity smoothing,␣
↪odometry, and safety services.
 *
 * This node provides:
 * - /cmd_vel subscriber with velocity smoothing
 * - CAN bus communication for wheel velocities and enable/disable
 * - Odometry publishing and TF
 * - /enable_cmd_vel, /hard_stop, /estop, /reset_odometry services
 * - Professional safety logic (E-stop lockout, smooth stopping, thread safety)
 *
 * @author Your Name
 * @date 2025-06-18
 */

#pragma once

#include <rclcpp/rclcpp.hpp>
#include <geometry_msgs/msg/twist.hpp>
#include <nav_msgs/msg/odometry.hpp>
```

INDEX

V